



SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Symbiosis International (Deemed University)

(Established under section 3 of the UGC Act, 1956)

Re-accredited by NAAC with 'A' grade (3.58/4) | Awarded Category – I by UGC

Founder: Prof. Dr. S. B. Mujumdar, M. Sc., Ph. D. (Awarded Padma Bhushan and Padma Shri by President of India)

Assignment No. 15

Subject:	DevOps Lab
TCODE:	TE7950
Name of Student	Adarsh Jha
PRN No.	23070122261
Branch	CSE, Batch (2023-27), L4
Academic Year & Semester	25 – 26, Sem VII
Date of Performance	23 rd July, 2026
Title of Assignment:	Basic Infrastructure as Code (IaC) with Terraform
Theory:	Infrastructure as Code (IaC) is the practice of managing and provisioning infrastructure using configuration files instead of manual processes. Terraform is an open-source IaC tool developed by HashiCorp that enables users to define, provision, modify, and destroy infrastructure resources in a consistent and automated manner. In this experiment, Terraform was used with the Local provider to create, update, and destroy a local text file. The commands terraform init, terraform plan, terraform apply, and terraform destroy demonstrated Terraform's workflow of initializing the working directory, previewing infrastructure changes, applying the desired configuration, and safely removing managed resources. This approach ensures repeatable deployments, version control of infrastructure, and simplified infrastructure management. Commands Used • terraform --version – Checks whether Terraform is installed correctly and displays the installed version. • mkdir my-terraform-iac – Creates a new directory for storing the Terraform project files. • cd my-terraform-iac – Navigates to the Terraform project directory. • terraform init – Initializes the Terraform working directory by downloading the required providers and preparing the project. • terraform plan – Generates an execution plan to preview the infrastructure changes before applying them. • terraform apply – Applies the Terraform configuration and creates or

	updates the resources defined in the configuration file. terraform destroy – Removes all resources managed by the current Terraform configuration.
Implementation:	Step 1: Install Terraform Installed Terraform — Successfully installed Terraform on the Windows system and verified the installation using the terraform --version command. Step 2: Create the Project Directory Created Terraform Project Directory — Created a new project directory named my-terraform-iac and navigated into it to store the Terraform configuration files. Step 3: Create the Terraform Configuration File (main.tf) Created Terraform Configuration File — Created the main.tf configuration file and defined the local provider along with a local_file resource to create a text file using Terraform. Step 4: Initialize Terraform Initialized Terraform — Initialized the Terraform working directory using the terraform init command, which downloaded the required provider plugins and prepared the project for execution. Step 5: Plan the Changes Generated Terraform Execution Plan — Generated an execution plan using the terraform plan command to preview the changes Terraform would make. The plan indicated that one new resource (local_file.my_example_file) would be created with no modifications or deletions. Step 6: Apply the Changes Applied Terraform Configuration — Applied the Terraform configuration using the terraform apply command, which successfully created the example.txt file as defined in the main.tf configuration. The generated file was verified in the project directory. Step 7: Modify the Content and Re-apply Command: terraform plan Generated Updated Execution Plan — Modified the main.tf configuration and generated an updated execution plan using the terraform plan command. Terraform detected that the existing local_file resource would be replaced to apply the updated content. Command: terraform apply

Applied Updated Terraform Configuration — Applied the modified Terraform configuration using the terraform apply command. Terraform successfully recreated the local_file resource with the updated content.

Verified Updated File Content — Verified that the example.txt file was successfully updated with the new content after applying the modified Terraform configuration.

Step 8: Destroy the Resource

Command: `terraform destroy`

Destroyed Terraform Resource — Executed the terraform destroy command, confirmed the destruction by entering yes, and successfully removed the Terraform-managed local_file resource from the local system.

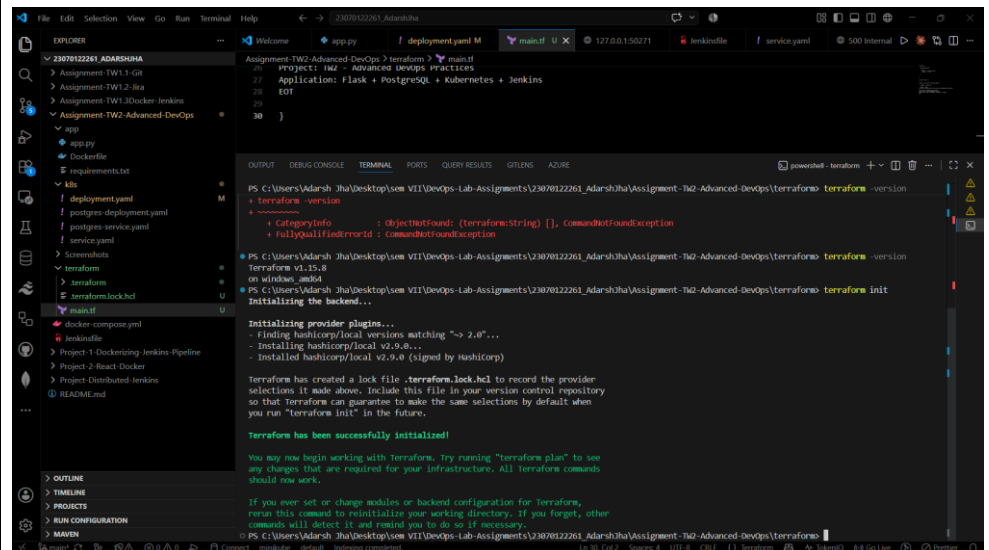
Verified Resource Deletion — Verified that the example.txt file was successfully removed from the project directory after the Terraform-managed resource was destroyed.

```
Administrator Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\WINDOWS\system32> choco install terraform
Chocolatey v2.5.1
Installing the following packages:
terraform
By installing, you accept licenses for the packages.
Downloading package from source 'https://community.chocolatey.org/api/v2/'
Progress: Downloading terraform 1.15.8... 100%

terraform v1.15.8 [Approved]
terraform package files install completed. Performing other installation steps.
The package terraform needs to run 'chocolateyinstall.ps1'.
Note: If you don't run this script, the installation will fail.
Note: To confirm automatically next time, use '-y' or consider:
choco features enable -n allowautomaticinstall
Do you want to run the script?([Y]es/[A]ll scripts/[N]o/[P]rint): y

Removing old terraform plugins
Downloading terraform 64 bit
from 'https://releases.hashicorp.com/terraform/1.15.8/terraform_1.15.8_windows_amd64.zip'
Progress: 100% - Completed download of C:\Users\Adarsh\AppData\Local\Temp\chocolatey\terraform\1.15.8\terraform_1.15.8_windows_amd64.zip (34.05 MB).
Download of terraform_1.15.8_windows_amd64.zip (34.05 MB) completed.
Bytes match.
Extracting C:\Users\Adarsh\AppData\Local\Temp\chocolatey\terraform\1.15.8\terraform_1.15.8_windows_amd64.zip to C:\ProgramData\chocolatey\lib\terraform\tools...
C:\ProgramData\chocolatey\lib\terraform\tools
Shadow has successfully created a shim for terraform.exe
The install of terraform was successful.
Installed to 'C:\ProgramData\chocolatey\lib\terraform\tools'
Chocolatey installed 1/1 packages.
See the log for details (C:\ProgramData\chocolatey\logs\chocolatey.log).
PS C:\WINDOWS\system32>
```



The screenshot shows the Visual Studio Code interface with a project named '23070122261_AdarshJha'. The Explorer sidebar on the left shows the project structure, including files like 'deployment.yaml', 'main.tf', 'terraform.lock.hcl', and 'terraform'. The main editor area displays the 'main.tf' file, which contains Terraform configuration for a Jenkins application. The bottom terminal window shows the output of the 'terraform init' command, indicating that the provider plugins are being initialized and the backend is being configured. The terminal output includes the following text:

```
PS C:\Users\Adarsh\AppData\Local\Temp\chocolatey\terraform\1.15.8> terraform -version
+ terraform -version
+ =====
+ GetpsycInfo : @psycNotFound: (terraform.String) [], CommandNotFoundException
+ nullPointerException

PS C:\Users\Adarsh\AppData\Local\Temp\chocolatey\terraform\1.15.8> terraform init
Initializing the backend...

Initializing provider plugins...
- Finding hashicorp/terraform-local versions matching ">= 2.0"...
- Installing hashicorp/terraform-local v2.0.0...
- Installed hashicorp/terraform-local v2.0.0 (signed by HashiCorp)

Terraform has created a lock file 'terraform.lock.hcl' to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.

PS C:\Users\Adarsh\AppData\Local\Temp\chocolatey\terraform\1.15.8>
```

```

PS C:\Users\Adarsh Jha\Desktop\sem VII\DevOps-Lab-Assignments\23070122261_AdarshJha\Assignment-TW2-Advanced-DevOps\terraform> terraform apply

Terraform Infrastructure as Code (IaC) Task Completed Successfully.

This file was created using Terraform.
Project: TW2 - Advanced DevOps Practices
Application: Flask + PostgreSQL + Kubernetes + Jenkins

EOT
+ content_base64sha256 = (known after apply)
+ content_base64sha512 = (known after apply)
+ content_md5 = (known after apply)
+ content_sha1 = (known after apply)
+ content_sha256 = (known after apply)
+ content_sha512 = (known after apply)
+ directory_permission = "0777"
+ file_permission = "0777"
+ filename = "output.txt"
+ id = (known after apply)
}

Plan: 1 to add, 0 to change, 0 to destroy.

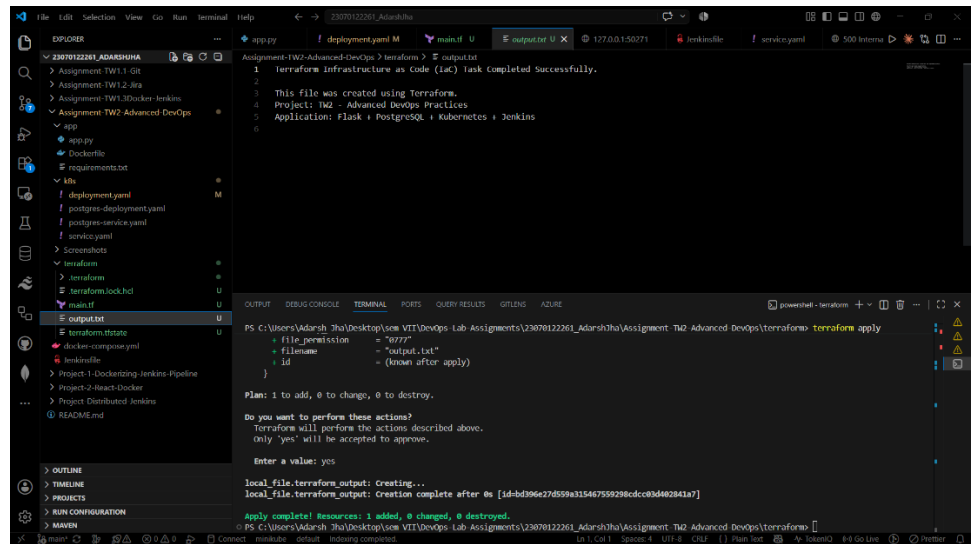
Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes

local_file.terraform_output: Creating...
local_file.terraform_output: Creation complete after 0s [id=bd396e27d559a315467559298cdc03d402841a7]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

```



```

C:\Users\Adarsh Jha\Desktop\sem VII\DevOps-Lab-Assignments\23070122261_AdarshJha\Assignment-TW2-Advanced-DevOps> terraform main.tf

1 terraform {
2
3   required_providers {
4
5     local = {
6       source = "hashicorp/local"
7       version = "~> 2.0"
8     }
9   }
10 }
11 }
12 }
13 }
14 }
15 provider "local" {}
16
17
18 resource "local_file" "terraform_output" {
19
20   filename = "output.txt"
21
22   content = <<EOT
23   Terraform Infrastructure as Code (IaC) Task Completed Successfully.
24
25   This file was created using Terraform.
26   Project: TW2 - Advanced DevOps Practices
27   Application: Flask + PostgreSQL + Kubernetes + Jenkins
28   EOT
29 }
30 }

```

